**UNIVERSIDADE FEDERAL DE OURO PRETO – UFOP**
Departamento de Computação – DECOM
Curso de Ciência da Computação

# Visualização Geométrica e Rasterização de Árvores Arteriais CCO

**Cauã Lemos Câmara**, **César Augusto Tiago Totô** e **Roberto Antonino Menegassi Filho**

Disciplina: BCC327 – Computação Gráfica Semestre: 2026.2
Docente: Prof. Dr. Rafael Alves Bonfim de Queiroz

**RESUMO**

Este trabalho apresenta o desenvolvimento parcial de uma aplicação para visualização de árvores arteriais geradas pelo método *Constrained Constructive Optimization* (CCO), utilizando modelos geométricos fornecidos em formato VTK. O objetivo foi desenvolver uma aplicação de Computação Gráfica que integre leitura, representação, transformação, rasterização e recorte de modelos CCO, destacando quais algoritmos clássicos foram implementados pelo grupo e quais foram delegados à biblioteca. A aplicação foi desenvolvida em Python com PyVista e reúne leitura dos dados, representação dos segmentos vasculares por tubos, codificação visual do raio, controles de câmera e projeção, preenchimento de polígono pelo algoritmo Scanline e seleção de técnicas de antialiasing disponibilizadas pela biblioteca. O modelo tridimensional `tree3D_Nterm016.vtk` foi utilizado como exemplo principal. Para esse arquivo, foram identificados 32 pontos, 31 células lineares e raios entre 0,2261241 e 0,6870107. O preenchimento Scanline aplicado a um polígono côncavo de teste ocupou 216 dos 780 pixels da grade (27,7%), conforme indicado pela própria execução. Os módulos de rasterização de linhas e de recorte de segmentos e polígonos permanecem sem implementação; além disso, o módulo de câmera oferece vistas predefinidas, mas não implementa controles próprios de translação, rotação e escala. Assim, o relatório distingue os recursos efetivamente implementados daqueles previstos no enunciado e ainda pendentes.

**Palavras-chave:** Computação Gráfica; PyVista; CCO; rasterização; Scanline; visualização 3D.

## 1 Introdução e objetivos

Árvores arteriais podem ser representadas computacionalmente como estruturas ramificadas compostas por pontos, segmentos e atributos geométricos associados aos vasos. No contexto deste trabalho, os modelos foram gerados previamente pelo método *Constrained Constructive Optimization* (CCO) e disponibilizados em arquivos VTK. O objetivo do TP1 não é reconstruir o método CCO, mas desenvolver uma aplicação que permita interpretar e visualizar seus resultados e exercitar conceitos fundamentais de Computação Gráfica.

A aplicação foi organizada em módulos Python e utiliza PyVista como interface de visualização sobre a infraestrutura VTK. O escopo efetivamente desenvolvido inclui a leitura dos modelos, a conversão de segmentos em tubos, a associação de cor e espessura ao raio, a seleção de vistas, a apresentação de uma região poligonal rasterizada por Scanline e a ativação de modos de antialiasing.

O objetivo geral deste artigo é documentar a arquitetura e os resultados observáveis da aplicação, com atenção à distinção entre algoritmos implementados pelo grupo e operações delegadas à biblioteca. Como objetivos específicos, busca-se: (i) ler e interpretar a estrutura geométrica dos modelos CCO (pontos, conectividade e raio); (ii) representar a árvore por tubos e codificar o raio por cor e/ou espessura; (iii) permitir vistas distintas do observador e a comparação entre projeção ortográfica e em perspectiva; (iv) implementar manualmente a rasterização de linhas (DDA e Bresenham) e de polígonos (Scanline); (v) comparar visualmente a renderização com e sem antialiasing; (vi) implementar o recorte de segmentos (Cohen-Sutherland e Liang-Barsky) e de polígonos (Sutherland-Hodgman); e (vii) documentar claramente o que foi implementado pelo grupo e o que foi realizado pela biblioteca.

O método CCO em si não foi reimplementado, os modelos foram fornecidos pelo professor.

# 2 Fundamentação teórica

## 2.1 Modelos vasculares CCO e representação geométrica

O método CCO é uma abordagem construtiva de otimização para geração de árvores vasculares, descrita por Schreiner e Buxbaum [1]. Neste TP1, os modelos CCO são dados de entrada fornecidos pelo docente. Portanto, a aplicação não executa o crescimento da árvore nem recalcula suas bifurcações: ela interpreta a geometria já armazenada no arquivo.

Os arquivos utilizados seguem o formato VTK `POLYDATA`. A estrutura contém coordenadas tridimensionais dos pontos, conectividade das células lineares e um atributo escalar denominado `raio`, armazenado por célula. Uma árvore pode, assim, ser tratada como um conjunto de vértices e arestas, em que cada aresta possui um raio associado. O número de pontos é obtido por `mesh.n_points`, enquanto o número de células é obtido por `mesh.n_cells`.

## 2.2 Primitivas gráficas e codificação do raio

Uma linha representa a conectividade entre dois pontos, mas pode ter pouca espessura visual. Para produzir uma representação volumétrica, o módulo 1 reconstrói cada segmento como uma linha independente e aplica o filtro `tube` do PyVista. O filtro gera uma superfície tubular ao redor da linha; no código, são solicitados oito lados e fechamento das extremidades (`capping=True`). A geração da superfície tubular é realizada pela biblioteca, não por um algoritmo de cilindrização implementado manualmente pelo grupo.

O raio é utilizado de duas maneiras independentes. Na codificação por cor, o valor de cada segmento é normalizado para o intervalo unitário:

$$t = \frac{r - r_{\min}}{r_{\max} - r_{\min}}, \qquad 0 \le t \le 1.$$

A cor RGB é então definida por

$$R = t, \qquad G = 0{,}5, \qquad B = 1 - t.$$

Dessa forma, valores menores de raio tendem ao azul e valores maiores tendem ao vermelho, enquanto o canal verde permanece constante. Na codificação por espessura, o raio de visualização é proporcional ao raio original, multiplicado por um fator de escala calculado a partir do comprimento da malha. Quando a opção é desativada, todos os segmentos recebem uma espessura visual constante baseada na média dos raios.

## 2.3 Câmera, vistas e projeção

A posição do observador determina como a geometria do mundo é apresentada na imagem. O espaço do mundo descreve as coordenadas dos objetos na cena; a transformação de visualização posiciona esses objetos em relação à câmera, definindo uma origem e orientações de referência para o observador. A projeção transforma as coordenadas no referencial da câmera em coordenadas de imagem.

Na projeção perspectiva, objetos mais distantes tendem a ocupar uma área menor na imagem. Na projeção ortográfica (ou paralela), linhas paralelas permanecem paralelas e o tamanho projetado não depende da distância à câmera. O módulo 2 oferece três posições predefinidas ($yz$, $xz$ e $xy$) e alterna o estado de projeção por meio das funções de câmera do PyVista. A rotação livre e o zoom também podem ser realizados pela interação padrão da janela de visualização.

### 2.4 Rasterização de Linhas

Rasterizar um segmento é escolher os pixels que melhor o aproximam. O **DDA - Digital Differential Analyzer** usa o passo passos = $\max(|\Delta x|, |\Delta y|)$ e incrementos ($\Delta x$/passos, $\Delta y$/passos) em ponto flutuante, arredondando cada posição. O **algoritmo de Bresenham** [5] usa apenas aritmética inteira. Para inclinação entre 0 e 1, mantém-se um parâmetro de decisão

$$p_0 = 2\Delta y - \Delta x, \qquad p_{k+1} = \begin{cases} p_k + 2\Delta y & \text{se } p_k < 0 \text{ (mantém } y) \\ p_k + 2(\Delta y - \Delta x) & \text{se } p_k \geq 0 \ (y \leftarrow y \pm 1) \end{cases} \tag{1}$$

e os demais octantes são tratados trocando os papéis de $x$ e $y$.

### 2.5 Rasterização de polígonos: Scanline Fill

Rasterizar um polígono significa determinar quais elementos de uma grade discreta pertencem ao seu interior. O Scanline Fill percorre a região linha a linha. Para cada coordenada vertical inteira, são calculadas as interseções da linha de varredura com as arestas não horizontais do polígono. As interseções são ordenadas pela coordenada horizontal e os pixels entre pares sucessivos são preenchidos, segundo a regra par–ímpar.

A implementação utiliza uma tabela de arestas (ET), organizada pela coordenada mínima $y$ de cada aresta, e uma tabela de arestas ativas (AET), atualizada a cada linha. A coordenada horizontal de cada interseção é atualizada incrementalmente por

$$x_{k+1} = x_k + \frac{1}{m},$$

em que $m$ é a inclinação da aresta. A implementação ignora arestas horizontais, remove arestas ao alcançar seu $y_{\max}$, ordena as interseções e preenche os intervalos delimitados por pares de interseções. O resultado é uma matriz binária: valor 1 para pixels preenchidos e 0 para pixels externos.

### 2.6 Aliasing e antialiasing

A rasterização de contornos inclinados pode produzir bordas visualmente serrilhadas, fenômeno conhecido como aliasing. Técnicas de antialiasing buscam reduzir esse efeito por meio de amostragem ou filtragem. O módulo 5 não implementa filtros próprios: ele oferece controles para desativar o antialiasing ou solicitar à biblioteca PyVista/VTK os modos MSAA, FXAA e SSAA. Assim, o trabalho demonstra a seleção dessas opções da API, sem apresentar uma implementação manual dos respectivos algoritmos.

**Tabela 1:** Comparação entre as técnicas de antialiasing (MSAA, FXAA e SSAA)

| **Técnica** | **Funcionamento** | **Qualidade** |
|---|---|---|
| **SSAA** (Super Sampling) | Renderiza a cena em uma resolução superior (ex: 4x) e depois redimensiona para a resolução nativa da tela. | **Altíssima**: Suaviza geometria, texturas e efeitos de shader em toda a cena. |
| **MSAA** (Multi-Sampling) | Amostra múltiplos pontos de cor e profundidade de forma seletiva, apenas nas *bordas dos polígonos*. | **Boa**: Excelente para arestas geométricas, mas não afeta texturas ou pixels internos. |
| **FXAA**(Fast Approximate) | Técnica de pós-processamento 2D que analisa a imagem já renderizada, deteta contrastes e desfoca as bordas. | **Aceitável**: Corrige o serrilhado de forma geral, mas pode causar um leve desfoque na imagem. |

### 2.7 Recorte de segmentos

**Cohen-Sutherland.** Cada extremo recebe um código de 4 bits (esquerda, direita, abaixo, acima). Se $c_1 \,|\, c_2 = 0$, o segmento é aceito; se $c_1 \,\&\, c_2 \neq 0$, é rejeitado; caso contrário, um extremo externo é substituído pela interseção com a borda indicada, por exemplo

$$x = x_1 + (x_2 - x_1)\,\frac{y_{\max} - y_1}{y_2 - y_1}, \quad y = y_{\max}, \tag{2}$$

e o processo se repete.

**Liang–Barsky** [6]. Parametriza-se $P(t) = P_1 + t(P_2 - P_1)$, $t \in [0, 1]$, e impõe-se $t\,p_k \leq q_k$ para cada borda, com $p = (-\Delta x, \Delta x, -\Delta y, \Delta y)$ e $q = (x_1 - x_{\min},\, x_{\max} - x_1,\, y_1 - y_{\min},\, y_{\max} - y_1)$. Para $p_k < 0$ o segmento está entrando e $r_k = q_k/p_k$ atualiza $t_{\text{ent}} = \max(0, r_k)$; para $p_k > 0$ está saindo e $t_{\text{sai}} = \min(1, r_k)$. Se $p_k = 0$ e $q_k < 0$, o segmento é paralelo e externo. O segmento é visível se $t_{\text{ent}} \leq t_{\text{sai}}$.

**Cyru–Beck** [7] (apenas discutido). Generaliza o problema para qualquer janela *convexa*: para cada aresta $i$ com normal externa $\mathbf{n}_i$ e ponto $\mathbf{P}_{E_i}$, impõe-se $\mathbf{n}_i \cdot (P(t) - \mathbf{P}_{E_i}) \leq 0$, o que dá $p_i = \mathbf{n}_i \cdot \mathbf{D}$ e $q_i = \mathbf{n}_i \cdot (\mathbf{P}_{E_i} - \mathbf{P}_1)$, com $\mathbf{D} = P_2 - P_1$. Liang–Barsky é a especialização desse esquema para retângulos alinhados aos eixos, onde os produtos escalares se reduzem a $\pm\Delta x$ e $\pm\Delta y$.

### 2.8 Recorte de polígonos

O algoritmo de **Sutherland–Hodgman** [8] recorta o polígono contra uma borda da janela por vez. Para cada aresta $(S, P)$ da lista de vértices, há quatro casos: (i) ambos dentro: emite $P$; (ii) $S$ dentro e $P$ fora: emite a interseção; (iii) ambos fora: nada; (iv) $S$ fora e $P$ dentro: emite a interseção e $P$. A saída de uma borda é a entrada da seguinte. O **Weiler–Atherton** [9] percorre os contornos do polígono e da janela alternando entre eles nos pontos de interseção, o que lhe permite produzir *vários* polígonos de saída.

## 3 Metodologia e arquitetura da aplicação

### 3.1 Tecnologias e organização

A aplicação foi escrita em Python e utiliza PyVista para leitura dos arquivos VTK, construção de objetos geométricos, renderização e widgets de interação. NumPy é utilizado nas operações numéricas sobre os atributos e na construção da grade de pixels; Matplotlib fornece o objeto de mapa de cores RGB utilizado pelo módulo 1. O ponto de entrada é `main.py`, que instancia um `Plotter` com janela de 1920 por 1080 pixels e chama sequencialmente os módulos.

**Tabela 2:** Responsabilidade e situação dos módulos da aplicação.

| Módulo | Tema | Comportamento observado no código | Situação |
|---|---|---|---|
| 1 | Leitura e atributos | Lê VTK, extrai pontos, células e raios; constrói tubos e controles de cor/espessura. | Implementado |
| 2 | Câmera e vistas | Oferece três vistas predefinidas e alternância de projeção pela API. | Parcial |
| 3 | Rasterização de linhas | A função `rasterizacao_linhas` retorna sem executar algoritmo. | Implementado |
| 4 | Rasterização de polígonos | Implementa ET/AET e preenchimento Scanline de polígono. | Implementado |
| 5 | Antialiasing | Widgets selecionam opções MSAA, FXAA, SSAA ou desativação da API. | Via biblioteca |
| 6 | Recorte de segmentos | A função `clipping_line_region` retorna sem executar algoritmo. | Implementado |
| 7 | Recorte de polígonos | A função `clipping_polygon_region` retorna sem executar algoritmo. | Implementado |

### 3.2 Leitura e preparação dos modelos

O módulo 1 carrega o arquivo informado na linha de comando ou, na ausência de argumento, utiliza `tree3D_Nterm016.vtk`. O nome do atributo de raio é localizado procurando-se por nomes que contenham `raio`, `radius` ou `radii`. Os valores mínimo e máximo são calculados com NumPy. Em seguida, cada célula é percorrida para recuperar os pares de pontos consecutivos, associando-se a cada segmento o raio da célula de origem.

O código cria uma nova estrutura `PolyData` com linhas de dois pontos e replica o raio nos dois extremos de cada segmento. Essa estrutura é encaminhada ao filtro `tube`. A interface disponibiliza caixas de seleção independentes para ligar ou desligar o mapa de cores e a espessura proporcional ao raio.

### 3.3 Câmera e interface

O módulo 2 cria dois grupos de botões de opção. O primeiro altera a projeção por meio de `enable_parallel_projection()` e `disable_parallel_projection()`, o segundo seleciona as posições `yz`, `xz` e `xy`. A interação de mouse permanece sob responsabilidade do visualizador PyVista/VTK.

Foi identificada uma inconsistência que deve ser considerada na interpretação das capturas: no código fornecido, a opção rotulada como "Projeção ortogonal" chama `disable_parallel_projection()`, enquanto a opção rotulada como "Projeção Paralela" chama `enable_parallel_projection()`. Como a projeção paralela corresponde à ortográfica, os rótulos da interface estão invertidos em relação às chamadas realizadas. Por isso, o relatório não atribui o tipo de projeção apenas ao texto selecionado na captura.

### 3.4 Rasterização de linhas

Os algoritmos DDA e Bresenham foram implementados, ambos retornando a lista de pixels $(x, y)$ do segmento. A versão de Bresenham trata todos os octantes: quando $|\Delta x| \geq |\Delta y|$ avança-se em $x$, e caso contrário em $y$, ajustando os sinais dos passos. Os pontos da árvore são levados à grade inteira por $\lceil 500\,c \rceil$ por coordenada, e os 20 primeiros segmentos são rasterizados. A figura de comparação marca os pixels de Bresenham (células coloridas), os centros dos pixels do DDA (círculos) e a reta ideal contínua (tracejada).

### 3.5 Rasterização de polígonos

Cada segmento da árvore é transformado em um retângulo de 4 vértices: dada a direção unitária $\mathbf{d}$ do segmento e sua normal $\mathbf{n}$, os vértices são $P_1 \mp \mathbf{n}h$ e $P_2 \pm \mathbf{n}h$ (com as pontas estendidas em $h$ para evitar buracos nas junções), onde a meia-largura $h$ é proporcional ao raio, com mínimo de 1 pixel. Os polígonos são projetados para um buffer de $400 \times 400$ pixels (o centro do modelo vai para o meio da imagem) e preenchidos com o Scanline (ET, AET e regra par-ímpar), com a cor definida pela mesma regra do Módulo 1. Os vasos são ordenados por profundidade $z$ (e, em empate, do maior para o menor raio) antes do desenho, aproximando o algoritmo do pintor. O módulo também implementa uma rotação do modelo em torno de seu centro (eixo $Y$ em 3D e eixo $Z$ em 2D) antes da projeção. O buffer é exibido como uma malha de pixels (`ImageData`) ao lado da árvore e pode ser ocultado por um botão.

### 3.6 Antialiasing

Um grupo de botões de rádio permite alternar entre ausência de antialiasing, MSAA, FXAA e SSAA, que são implementados pelo pipeline gráfico do VTK e acionados via `enable_anti_aliasing` do PyVista. Este módulo não implementa os algoritmos, sua contribuição é a comparação interativa e a explicação do seu funcionamento.

### 3.7 Recorte de segmentos

Cohen-Sutherland e Liang-Barsky foram implementados manualmente, ambos retornando se o segmento é visível e seus extremos recortados. Contadores registram o número de interseções e de divisões realizadas, para a comparação entre os métodos. A janela de recorte é um retângulo centrado na árvore, com tamanho controlado por um *slider* (de 10% a 120% da extensão do modelo; padrão de 50%). O recorte opera no plano $xy$, em modelos 3D, o valor de $z$ dos novos extremos é obtido por interpolação linear ao longo do segmento original. A cena exibe a janela (laranja), as partes mantidas (verde) e as descartadas (vermelho), e um painel com o total de segmentos aceitos inteiros, cortados e rejeitados. Há também a geração de uma figura com três painéis (antes, depois com Cohen-Sutherland e depois com Liang-Barsky).

### 3.8 Recorte de polígonos

O Sutherland-Hodgman foi implementado para janelas retangulares alinhadas aos eixos, em quatro funções. `is_inside` testa o semiplano de cada borda, considerando pontos sobre a borda como internos. `compute_intersection` calcula o ponto de cruzamento por parametrização da aresta e, se ela é paralela à borda, devolve o ponto inicial. `sutherland_hodgman_clip_edge` realiza um passo do algoritmo: percorre os vértices tomando o último como "anterior" do primeiro e aplica os quatro casos da Seção 2. Por fim, `sutherland_hodgman_clip` encadeia as quatro bordas na ordem *abaixo, direita, topo, esquerda*, a saída de cada etapa servindo de entrada da seguinte, e devolve também o histórico do polígono após cada borda. Cada etapa percorre a lista uma única vez, portanto o custo é linear no número de vértices.

O polígono de teste usado na demonstração é **convexo**, com 7 vértices: $(-7,-2)$, $(-3,-6)$, $(4,-6)$, $(8,-1)$, $(6,5)$, $(0,8)$ e $(-5,6)$. Ele fica na constante `POLIGONO_TESTE`, o que permite trocá-lo por qualquer outra lista de vértices. A janela de recorte tem tamanho fixo de $10 \times 8$, inicialmente centrada em (3,5; 2,5), e é **arrastada com o botão esquerdo do mouse**: observadores do VTK tratam os eventos de pressionar, mover e soltar, e as coordenadas de tela são convertidas para o mundo por `DisplayToWorld`. O centro é limitado a $[-15,15] \times [-11,11]$ para a janela não sair da cena.

A cena é 2D, com projeção paralela, câmera sobre o eixo $z$ e estilo de interação de imagem (sem rotação, apenas zoom). O polígono original é desenhado em cinza, a janela em preto e a saída do algoritmo em vermelho, em três níveis de $z$ ligeiramente diferentes para evitar sobreposição visual. A

cada movimento da janela o recorte é recalculado sobre o polígono inteiro, e a saída só é desenhada se tiver pelo menos 3 vértices.

# 4 Resultados

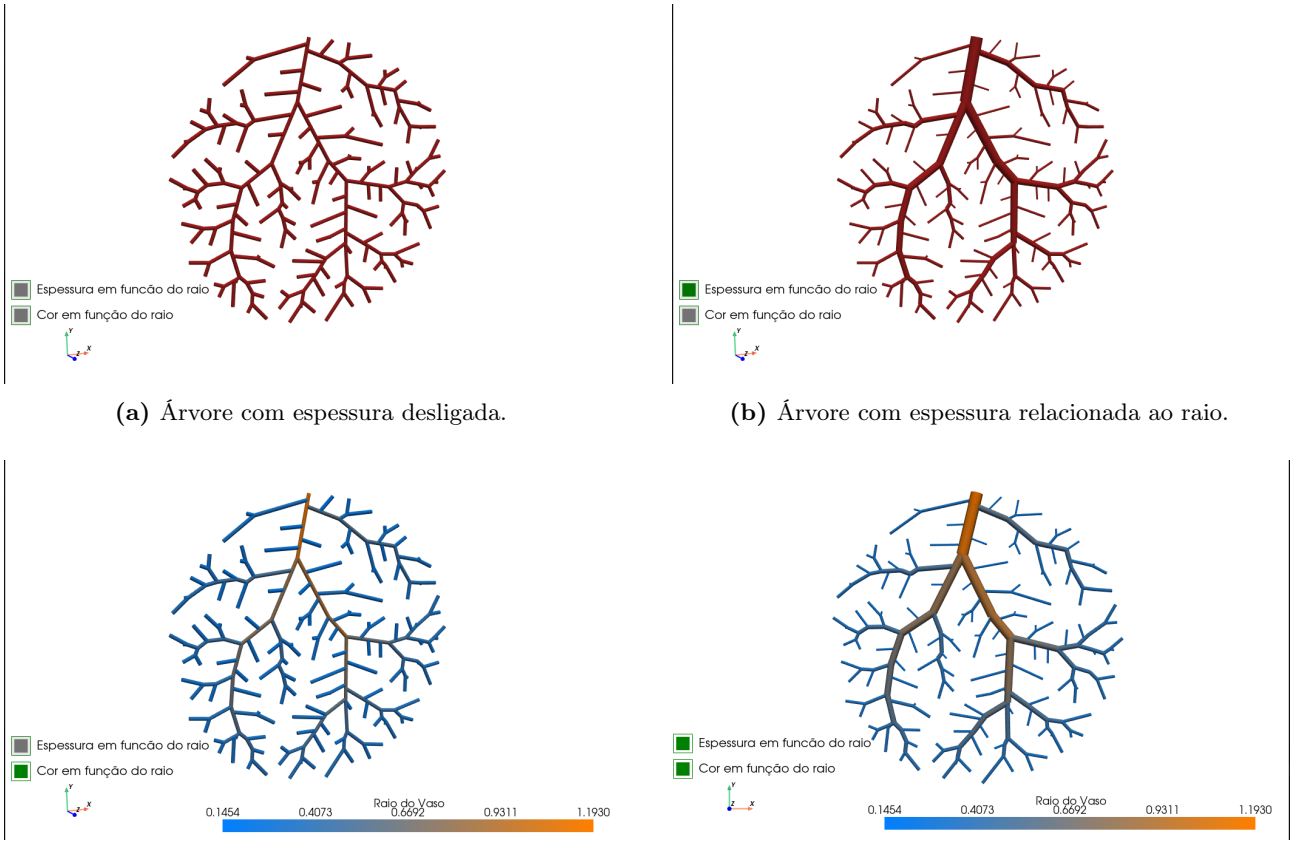
Nos testes foram utiliza dois modelos de árvore tree2D_Nterm128.vtk e tree3D_Nterm016.vtk.

## 4.1 Leitura e representação do modelo

A Tabela 3 resume o modelo utilizado nos testes e a Figura 1 mostra as árvore representadas por tubos.

**Tabela 3:** Características do modelo CCO utilizado.

| **Atributo** | **Valor** |
|---|---|
| Arquivo | tree2D_Nterm128.vtk |
| Dimensão | 2D |
| Número de pontos | 256 |
| Número de segmentos | 255 |
| Raio mínimo | 0.1454243 |
| Raio máximo | 1.1929671 |




**(a)** Árvore com espessura desligada.

**(b)** Árvore com espessura relacionada ao raio.

**(c)** Árvore com espessura relacionada com a cor.

**(d)** Árvore com espessura relacionada à cor e ao raio.

**Figura 1:** Comparação das diferentes configurações da árvore.

## 4.2 Modelo utilizado e atributos

O modelo de exemplo é `tree3D_Nterm016.vtk`. A leitura direta do arquivo e a informação apresentada pela aplicação indicam 32 pontos e 31 células lineares. O atributo `raio` possui mínimo de 0,2261241 e máximo de 0,6870107. Esses valores são reproduzidos na interface durante a execução (Figura 2).

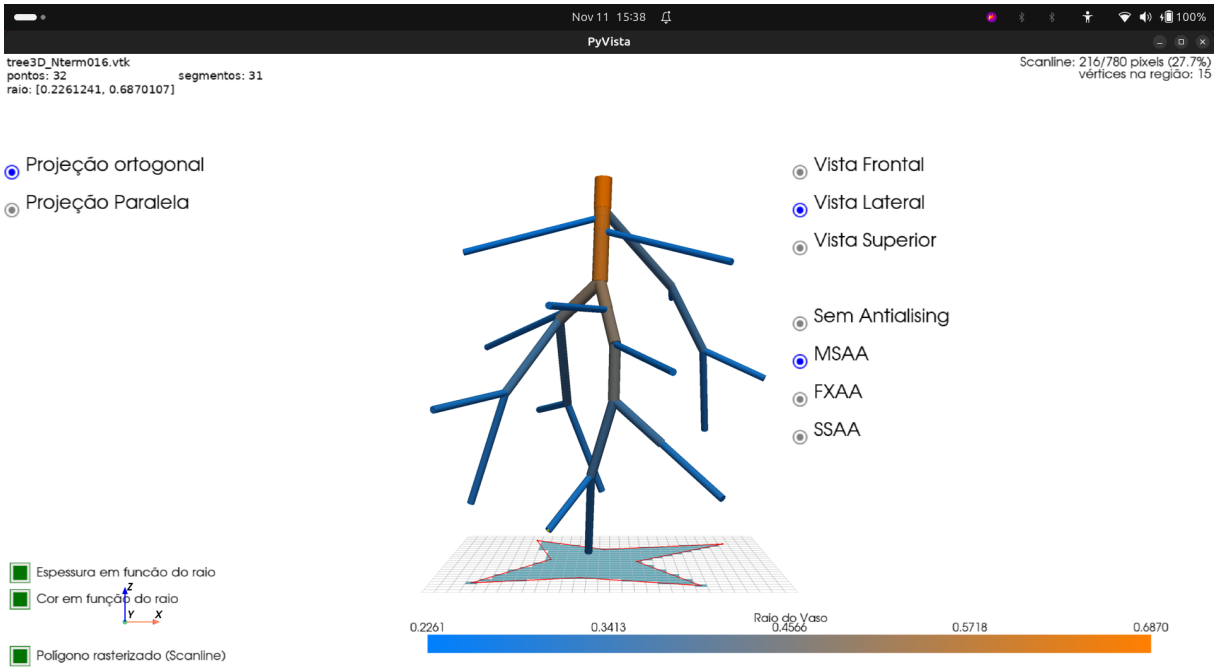
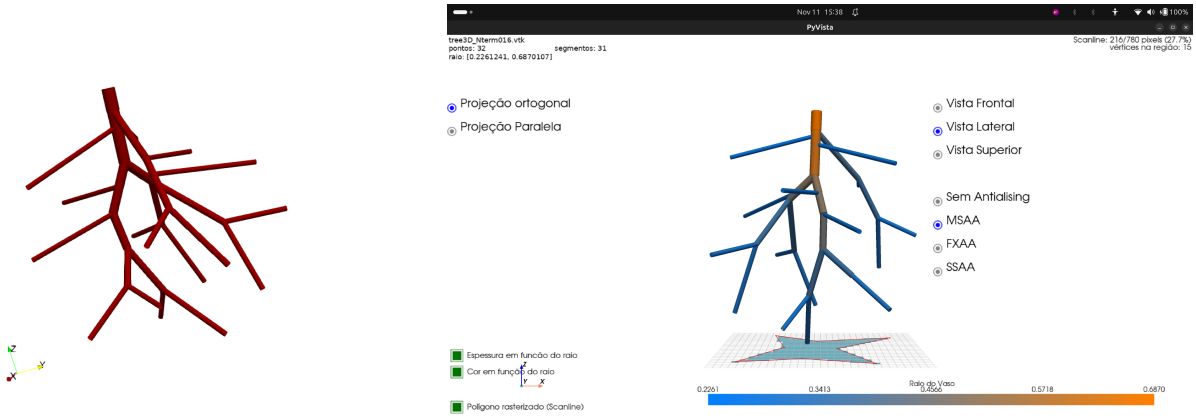



**Figura 2:** Aplicação em execução com o modelo `tree3D_Nterm016.vtk`, mostrando os dados de entrada, a árvore tubular, os controles de câmera, antialiasing e atributos visuais.

A Figura 3 compara a imagem de referência fornecida pelo docente para o mesmo conjunto com uma captura da aplicação. A comparação é qualitativa: ambas apresentam a estrutura ramificada, mas a aplicação adiciona controles interativos e a codificação visual pelo atributo de raio.

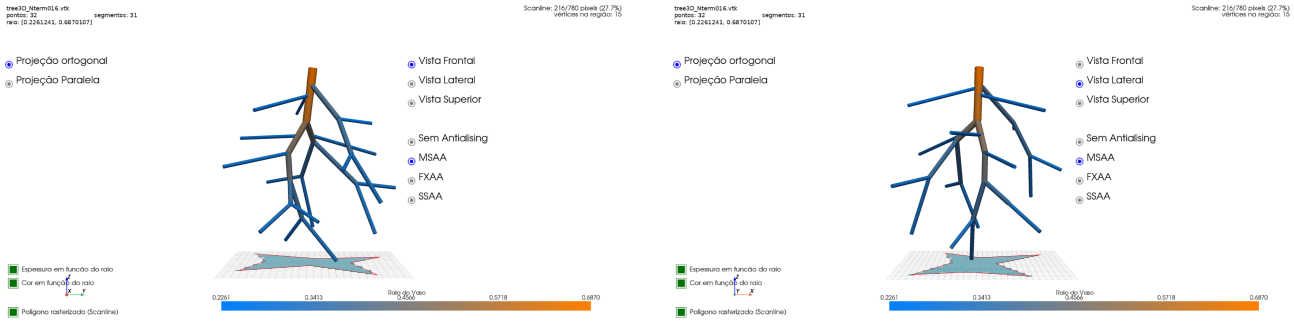


**(a)** Imagem de referência do docente.

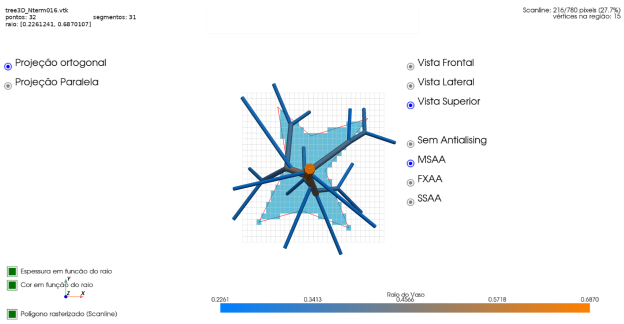
**(b)** Visualização produzida pela aplicação.

**Figura 3:** Comparação qualitativa do modelo tridimensional com a referência disponibilizada.

Na Figura 4 mostra a comparação da visualização da árvore ao mudar a câmera de posição.




**(a)** Frontal. **(b)** Lateral.




**(c)** Superior.

**Figura 4:** Comparação da visualização da árvore ao mudar a posição da câmera.

## 4.3 Rasterização de linhas

Na Figura 5 mostra a imagem gerada pela execução do algoritmo de Rasterização de Linha DDA e Bresenham.

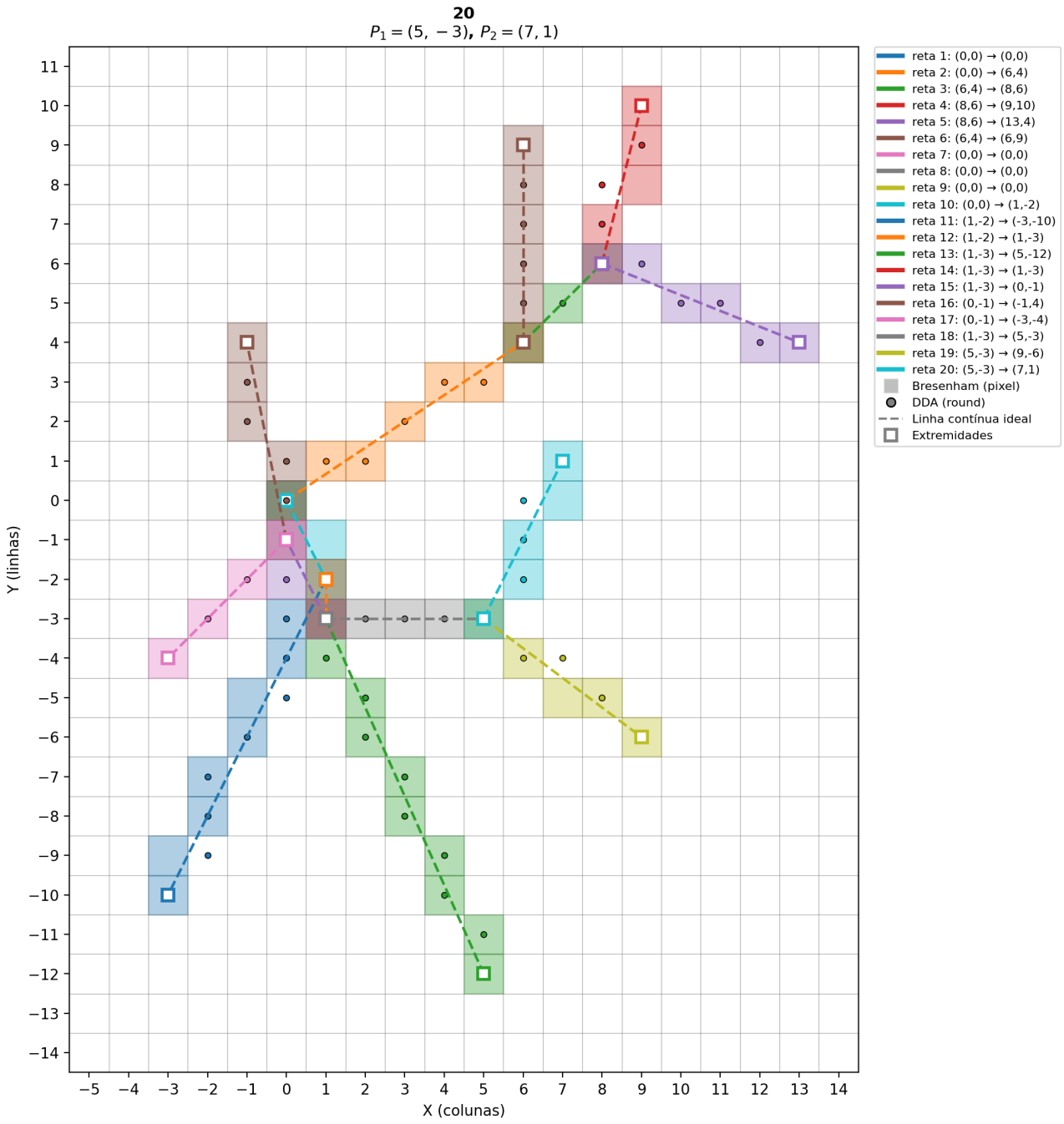
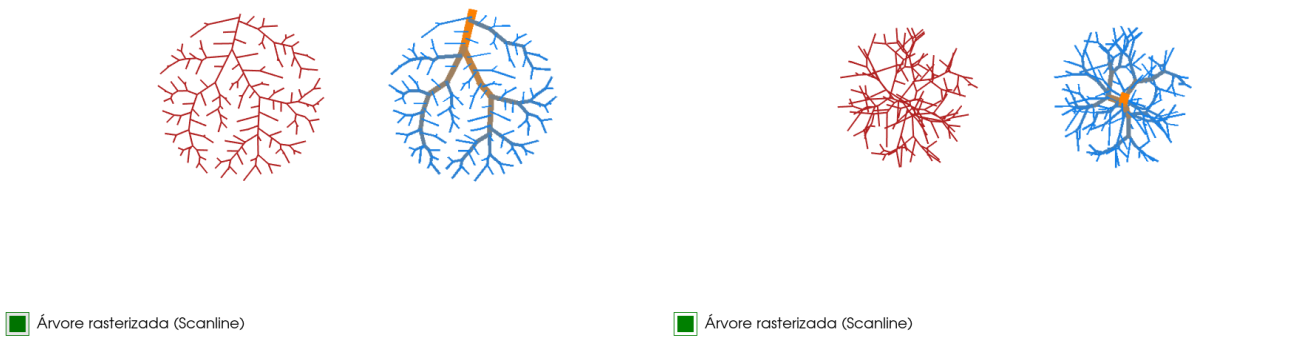



**Figura 5:** Rasterização de Linhas.

## 4.4 Rasterização de polígonos

Na Figura 6 mostra a comparação da rasterização de polígono (algoritmo Scanline) da árvore em dimensões diferentes do modelo do arquivo treexD_Nterm128.vtk. A árvore a direita em azul é a árvore rasterizada.

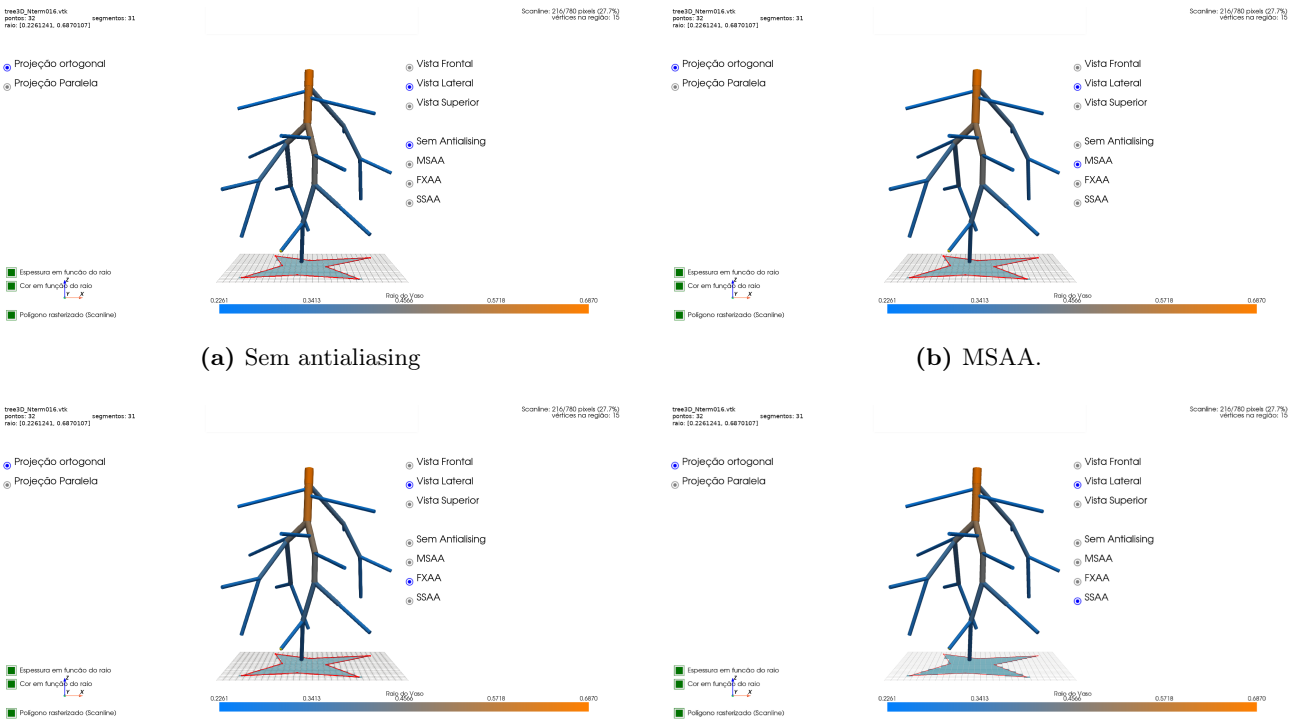



**(a)** Rasterização de Polígono em Árvore 2D. **(b)** Rasterização de Polígono em Árvore 3D.

**Figura 6:** Comparação de Rasterização em diferentes dimensões de árvore.

## 4.5 Antialiasing

Na figura 7 mostra a alternância entre ausência de antialiasing, MSAA, FXAA e SSAA.




**(a)** Sem antialiasing **(b)** MSAA.

**(c)** FXAA. **(d)** SSAA.

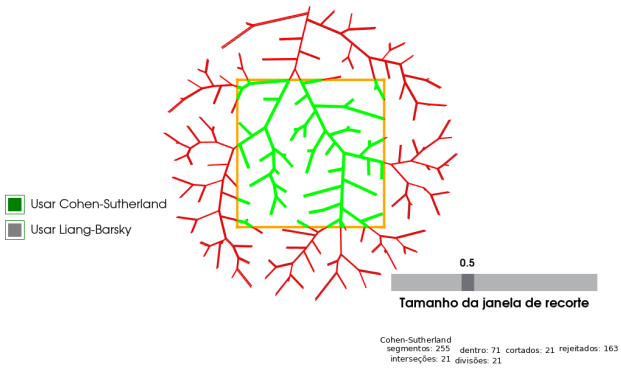
**Figura 7:** Comparação das diferentes configurações de Antialiasing.

## 4.6 Recorte de segmentos

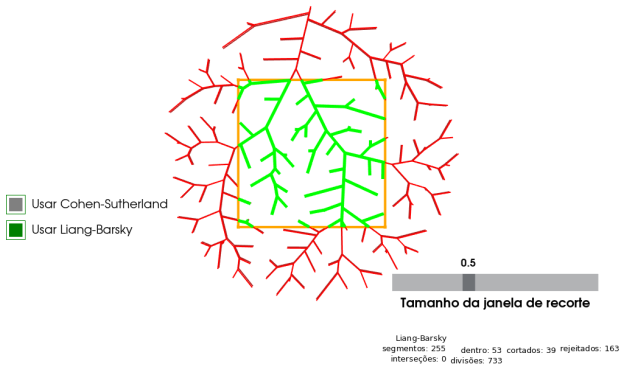
A Tabela 4 compara os algoritmos para a janela com fator de 50% do tamanho da árvore para o modelo tree2D_Nterm128.vtk. E a Figura 8 mostra visualmente a comparação da execução dos algoritmos.

**Tabela 4:** Estatísticas do recorte, obtidas dos contadores da aplicação.

| Algoritmo | Segmentos | Dentros | Cortados | Rejeitados | Interseções / Divisões |
|---|---|---|---|---|---|
| Cohen-Sutherland | 255 | 71 | 21 | 163 | 21 / 21 |
| Liang-Barsky | 255 | 53 | 39 | 163 | 0 / 733 |




**(a)** Execução do algoritmo Cohen-Sutherland.




**(b)** Execução do algoritmo Liang-Barsky.

**Figura 8:** Comparação da execução dos algoritmos Cohen-Sutherland e Liang-Barsky.

### 4.7 Recorte de polígonos

Na Figura 9 mostra o resultado da execução do algoritmo Sutherland-Hodgamn no recorte de polígonos.

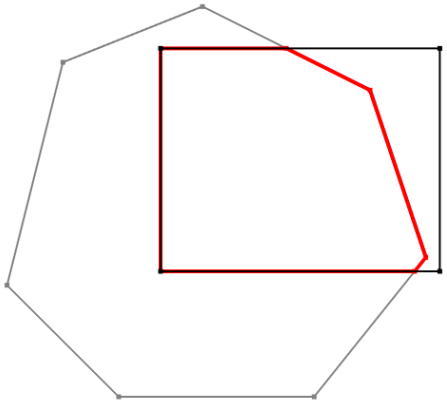


**Figura 9:** Recorte de Polígono.

## 5 Discussão

Os resultados demonstram a integração de diferentes conceitos em uma aplicação única. O módulo 1 realiza a interpretação do arquivo VTK e constrói uma representação tubular que torna a estrutura vascular mais legível. A codificação do raio por cor e espessura amplia a quantidade de informação visual sem modificar a geometria topológica de entrada. O módulo 4 constitui a parte algorítmica

mais explícita: o preenchimento é calculado em código próprio, com tabela de arestas e atualização incremental das interseções, em vez de ser delegado integralmente ao renderizador.

Há, entretanto, diferenças importantes entre os requisitos do enunciado e o estado atual do programa. O módulo 2 permite escolher vistas e alternar a projeção por meio da API, mas não implementa operações explícitas de translação, rotação e escala do objeto. A interface também apresenta os rótulos das projeções invertidos em relação às chamadas da câmera.

### 5.1 Implementado pelo grupo × auxílio da biblioteca

A Tabela 5 separa explicitamente as partes.

**Tabela 5:** Divisão entre o que foi implementado pelo grupo e o que foi fornecido pelo PyVista/VTK.

| **Tópico** | **Implementado pelo grupo** | **Auxílio de biblioteca** |
|---|---|---|
| 1. Leitura/atributos | Decomposição em segmentos, detecção do raio, mapa de cor, escala de espessura | Leitura do VTK; filtro de tubos |
| 2. Observador | Organização de vistas e botões | Câmera, matrizes de visão e projeção, interação |
| 3. Linhas | DDA e Bresenham | Figura com Matplotlib |
| 4. Polígonos | Scanline (ET/AET), geometria dos retângulos, projeção, rotação | Exibição do buffer como malha de pixels |
| 5. Antialiasing | Interface de seleção e comparação | MSAA, FXAA e SSAA |
| 6. Recorte de segmentos | Cohen-Sutherland, Liang-Barsky, interpolação de $z$, contadores | Exibição das linhas |
| 7. Recorte de polígonos | Sutherland-Hodgman (quatro bordas, histórico por etapa), polígono de teste, arraste da janela | Janela 2D, câmera paralela, eventos de mouse (VTK) |

### 5.2 Comparação entre os algoritmos de recorte

Cohen-Sutherland é eficiente quando a maioria dos segmentos é trivialmente aceita ou rejeitada, mas, para segmentos que cruzam a janela, pode recalcular interseções e códigos por até quatro iterações, cada uma com uma divisão. Liang-Barsky calcula no máximo quatro razões $q_k/p_k$ e só obtém os pontos recortados uma vez ao final, evitando interseções intermediárias. Cyrus-Beck, não implementado, usa a mesma ideia paramétrica, mas aceita qualquer janela convexa, ao custo de produtos escalares em vez de simples diferenças de coordenadas.

### 5.3 Sutherland-Hodgman e Weiler-Atherton

Sutherland-Hodgman funciona com qualquer polígono contra uma janela convexa, mas, ao recortar polígonos côncavos, pode produzir arestas degeneradas (sobrepostas) ao ligar partes que deveriam ser regiões separadas, pois sua saída é sempre uma única lista de vértices. O Weiler-Atherton [9] é mais adequado quando há polígonos côncavos, quando a interseção gera múltiplas regiões disjuntas e quando a janela também é um polígono arbitrário, incluindo côncavo ou com buracos: ao percorrer as listas de entrada e saída, ele separa corretamente as regiões resultantes. Em contrapartida, é bem mais complexo de implementar.

### 5.4 Decisões de implementação e limitações

- O mapa de cores RGB linear é simples, mas não é perceptualmente uniforme; mapas contínuos como o *viridis* facilitariam a leitura de diferenças sutis de raio.

- No DDA, a função `round` de Python arredonda valores intermediários ($x$,5) para o inteiro par mais próximo, o que pode gerar pequenas diferenças em relação ao Bresenham.
- O Módulo 3 rasteriza apenas os 20 primeiros segmentos, por clareza visual; a fração 500 de conversão para inteiros depende da escala do modelo.
- O recorte do Módulo 6 considera apenas o plano $xy$ (com interpolação de $z$); não há recorte volumétrico em 3D.
- No Módulo 4, a ordenação por profundidade é uma aproximação (algoritmo do pintor) e não substitui um Z-buffer.
- No Módulo 7, a janela só pode ser arrastada (tamanho fixo), a interface demonstra apenas o polígono convexo (o côncavo foi testado por chamada direta da função) e o módulo abre janela própria.
- O Sutherland-Hodgman implementado serve apenas a janelas retangulares alinhadas aos eixos e não separa regiões disjuntas.

# 6 Conclusão

Este trabalho resultou em uma aplicação modular que integra leitura, visualização, rasterização, antialiasing e recorte de modelos de árvores arteriais CCO. Os algoritmos clássicos solicitados (DDA e Bresenham, Scanline, Cohen-Sutherland, Liang-Barsky e Sutherland-Hodgman) foram implementados diretamente pelo grupo, enquanto a leitura dos arquivos, a renderização, a câmera e o antialiasing ficaram a cargo do PyVista/VTK, como documentado na Tabela 5.

Os experimentos permitiram observar, de forma concreta, o que os livros descrevem: DDA e Bresenham geram praticamente os mesmos pixels com custos diferentes; Cohen-Sutherland e Liang-Barsky recortam igualmente os segmentos, mas com quantidades distintas de interseções e divisões; e o Sutherland-Hodgman é simples e correto para polígonos convexos, mas não representa corretamente a saída quando o recorte de um polígono côncavo produz mais de uma região. O trabalho também deixou claro o contraste entre algoritmos didáticos e o pipeline gráfico, que realiza essas etapas em hardware.

# Agradecimentos

Os autores agradecem ao Departamento de Computação (DECOM) e à Universidade Federal de Ouro Preto (UFOP) pela infraestrutura e ao Prof. Rafael Alves Bonfim de Queiroz pelo material, aulas e pelos modelos CCO fornecidos.

## Referências

[1] SCHREINER, W.; BUXBAUM, P. Computer-optimization of vascular trees. *IEEE Transactions on Biomedical Engineering*, v. 40, n. 5, p. 482–491, 1993.

[2] FOLEY, J. D.; VAN DAM, A.; FEINER, S. K.; HUGHES, J. F. *Computer Graphics: Principles and Practice*. 2. ed. Reading: Addison-Wesley, 1996.

[3] PYVISTA DEVELOPERS. *PyVista Documentation*. Disponível em: https://docs.pyvista.org/. Acesso em: 11 nov. 2026.

[4] KITWARE. *VTK – The Visualization Toolkit*. Disponível em: https://vtk.org/. Acesso em: 11 nov. 2026.

[5] BRESENHAM, J. E. Algorithm for computer control of a digital plotter. **IBM Systems Journal**, v. 4, n. 1, p. 25–30, 1965.

[6] LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. **ACM Transactions on Graphics**, v. 3, n. 1, p. 1–22, 1984.

[7] CYRUS, M.; BECK, J. Generalized two- and three-dimensional clipping. **Computers & Graphics**, v. 3, n. 1, p. 23–28, 1978.

[8] SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. **Communications of the ACM**, v. 17, n. 1, p. 32–42, 1974.

[9] WEILER, K.; ATHERTON, P. Hidden surface removal using polygon area sorting. In: **SIGGRAPH '77**, p. 214–222, 1977.